

PATRÓN BUILDER

Patrón de Diseño: *Builder*

Creación y configuración de flujos de entrada y salida
(Reader/Writer) usando el patrón Builder

Autor: **Angie Nathaly Cisneros Guerrero**

20242020084

Fecha: Septiembre 2026

Índice

1. Diagrama UML	2
2. Explicación de las Clases	3
2.1. Productos Finales	3
2.1.1. Clase BufferedReader	3
2.1.2. Clase BufferedWriter	3
2.2. Clase Abstracta Builder	5
2.3. Constructores Concretos	6
2.3.1. Clase BuilderReader	6
2.3.2. Clase BuilderWriter	7
2.4. Clase Director	9
3. Código de Cliente y Explicación	10
3.1. Clase Cliente (Main)	10
4. Pruebas	12
4.1. Prueba de Lectura (BufferedReader)	12
4.2. Prueba de Escritura (BufferedWriter)	13
5. Estructura del Proyecto	14
5.1. Enlace al Repositorio	14
5.2. Organización de Paquetes	14

Diagrama UML

Descripción General

El proyecto implementa el patrón de diseño **Builder** para abstraer la compleja creación de objetos de entrada y salida de datos (I/O) en Java. Este patrón separa la construcción de los objetos `BufferedReader` y `BufferedWriter` de su representación, permitiendo que un mismo proceso de construcción (**Director**) cree diferentes representaciones según el constructor concreto utilizado.

Diagrama de Clases

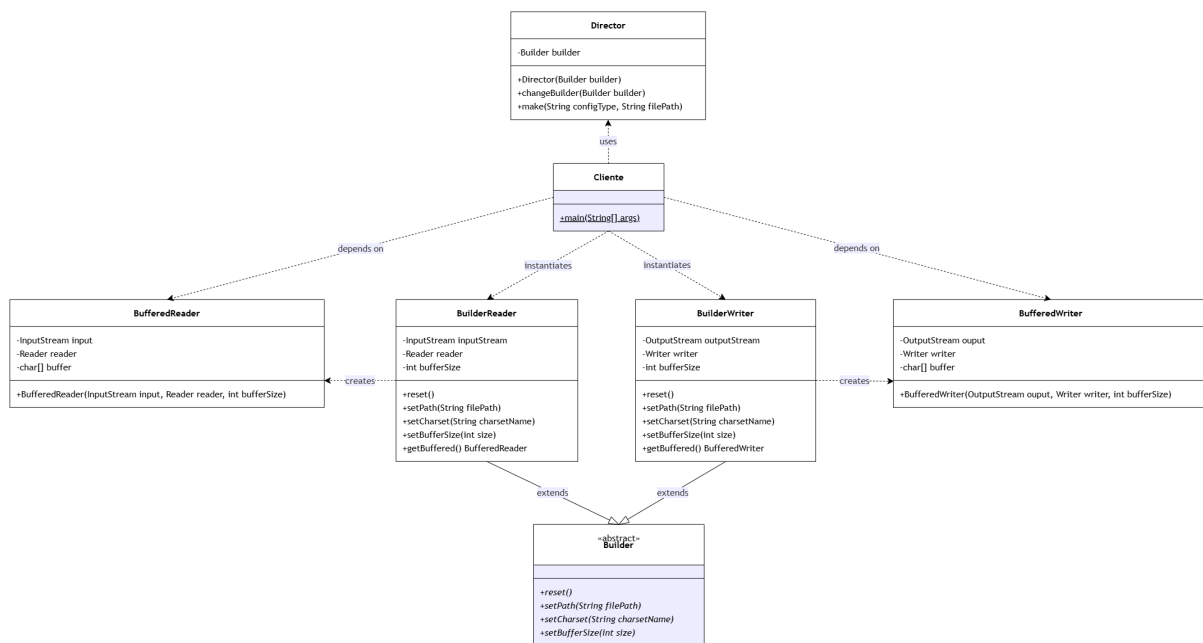


Figura 1: Diagrama UML del Patrón Builder aplicado a I/O.

Explicación de las Clases

Productos Finales

2.1.1. Clase BufferedReader

Descripción: La clase `BufferedReader` representa el producto final para la lectura de archivos, encapsulando el flujo de bytes (`InputStream`), el flujo de caracteres (`Reader`) y el arreglo que sirve como búfer de memoria[cite: 10].

Código Fuente:

```
1 package buffered.builder;
2
3 import java.io.IOException;
4 import java.io.InputStream;
5 import java.io.Reader;
6
7 public class BufferedReader {
8
9     private InputStream input;
10    private Reader reader;
11    private char[] buffer;
12
13    public BufferedReader(InputStream input, Reader reader, int
        bufferSize) {
14        this.input = input;
15        this.reader = reader;
16        this.buffer = new char[bufferSize];
17    }
18
19    public void imprimirContenido() throws IOException {
20        int caracter;
21        System.out.println("Iniciando lectura");
22        while ((caracter = this.reader.read()) != -1) {
23            System.out.print((char) caracter);
24        }
25        System.out.println("\nlectura completa");
26    }
27 }
```

Explicación:

- Contiene un método `imprimirContenido()` que itera sobre el `Reader` para imprimir los caracteres del archivo hasta llegar al final (-1)[cite: 10].

2.1.2. Clase BufferedWriter

Descripción: Representa el producto final para la escritura de datos, almacenando un `OutputStream`, un `Writer` y su respectivo búfer[cite: 11].

Código Fuente:

```
1 package buffered.builder;
2
3 import java.io.OutputStream;
4 import java.io.Writer;
5
6 public class BufferedWriter {
7
8     private OutputStream ouput;
9     private Writer writer;
10    private char[] buffer;
11
12    public BufferedWriter(OutputStream ouput, Writer writer, int
        bufferSize) {
13        this.ouput = ouput;
14        this.writer = writer;
15        this.buffer = new char[bufferSize];
16    }
17 }
```

Clase Abstracta Builder

Descripción: Define los pasos estándar que todos los constructores concretos deben implementar (`reset`, `setPath`, `setCharset` y `setBufferSize`)[cite: 12].

Código Fuente:

```
1 package buffered.builder;
2
3 public abstract class Builder {
4
5     public abstract void reset();
6     public abstract void setPath(String path) throws Exception;
7     public abstract void setCharset(String charsetName) throws
8         Exception;
9     public abstract void setBufferSize(int size);
10 }
```

Constructores Concretos

2.3.1. Clase BuilderReader

Descripción: Implementa la interfaz Builder para construir un `BufferedReader`. Utiliza internamente las clases nativas `FileInputStream` e `InputStreamReader` de Java para establecer la conexión[cite: 13].

Código Fuente:

```
1 package buffered.builder;
2 import java.io.FileInputStream;
3 import java.io.InputStream;
4 import java.io.InputStreamReader;
5 import java.io.Reader;
6
7 public class BuilderReader extends Builder {
8
9     private InputStream inputStream;
10    private Reader reader;
11    private int bufferSize;
12
13    @Override
14    public void reset() {
15        this.inputStream = null;
16        this.reader = null;
17        this.bufferSize = 8192;
18    }
19
20    @Override
21    public void setPath(String filePath) throws Exception {
22        this.inputStream = new FileInputStream(filePath);
23    }
24
25    @Override
26    public void setCharset(String charsetName) throws Exception {
27        if (this.inputStream == null) {
28            throw new IllegalStateException("definir el Path
29                primero");
30        }
31        this.reader = new InputStreamReader(this.inputStream,
32            charsetName);
33    }
34
35    @Override
36    public void setBufferSize(int size) {
37        if (size <= 0) throw new IllegalArgumentException("debe
38            ser mayor a 0");
39        this.bufferSize = size;
40    }
41
42    public BufferedReader getBuffered() {
43        if (this.reader == null && this.inputStream != null) {
```

```
41         this.reader = new InputStreamReader(this.inputStream)
42         ;
43     }
44     return new BufferedReader(this.inputStream, this.reader,
45         this.bufferSize);
46 }
```

2.3.2. Clase BuilderWriter

Descripción: Implementa la lógica para construir un `BufferedWriter`, utilizando `FileOutputStream` y `OutputStreamWriter`[cite: 14].

Código Fuente:

```
1 package buffered.builder;
2
3 import java.io.FileOutputStream;
4 import java.io.OutputStream;
5 import java.io.OutputStreamWriter;
6 import java.io.Writer;
7
8 public class BuilderWriter extends Builder {
9
10     private OutputStream outputStream;
11     private Writer writer;
12     private int bufferSize;
13
14     @Override
15     public void reset() {
16         this.outputStream = null;
17         this.writer = null;
18         this.bufferSize = 8192;
19     }
20
21     @Override
22     public void setPath(String filePath) throws Exception {
23         this.outputStream = new FileOutputStream(filePath);
24     }
25
26     @Override
27     public void setCharset(String charsetName) throws Exception {
28         if (this.outputStream == null) {
29             throw new IllegalStateException("definir el Path
30                 primero");
31         }
32         this.writer = new OutputStreamWriter(this.outputStream,
33             charsetName);
34     }
35 }
```

```
34     @Override
35     public void setBufferSize(int size) {
36         if (size <= 0) throw new IllegalArgumentException("debe_
37             ser_mayor_a_0");
38         this.bufferSize = size;
39     }
40
41     public BufferedWriter getBuffered() {
42         if (this.writer == null && this.outputStream != null) {
43             this.writer = new OutputStreamWriter(this.
44                 outputStream);
45         }
46         return new BufferedWriter(this.outputStream, this.writer,
47             this.bufferSize);
48     }
49 }
```

Clase Director

Descripción: El `Director` orquesta los pasos de construcción. Recibe cualquier objeto que herede de `Builder` y ejecuta las órdenes en el método `construir(type, path)` sin saber si está armando un lector o un escritor[cite: 16].

Código Fuente:

```
1 package buffered.builder;
2
3 public class Director {
4
5     private Builder builder;
6
7     public Director(Builder builder) {
8         this.builder = builder;
9     }
10
11     public void changeBuilder(Builder builder) {
12         this.builder = builder;
13     }
14
15     public void construir(String type, String path) throws
16         Exception {
17         builder.reset();
18
19         if (type.equalsIgnoreCase("simple")) {
20             builder.setPath(path);
21         } else {
22             builder.setPath(path);
23             builder.setBufferSize(16384);
24         }
25     }
26 }
```

Código de Cliente y Explicación

Clase Cliente (Main)

Descripción: La clase `Cliente` es el punto de entrada. Utiliza `JFileChooser` para seleccionar los archivos, instancia los constructores específicos (`BuilderReader` y `BuilderWriter`) y se los pasa al `Director` para que coordine la creación de los objetos finales[cite: 15].

Código Fuente:

```
1 package buffered.builder;
2
3 import java.io.File;
4 import javax.swing.JFileChooser;
5
6 public class Cliente {
7
8     public static void main(String[] args) {
9
10         JFileChooser selector = new JFileChooser();
11         selector.setDialogTitle("archivo_de_entrada_reader");
12         int seleccionLectura = selector.showOpenDialog(null);
13
14         if (seleccionLectura == JFileChooser.APPROVE_OPTION) {
15             File archivoEntrada = selector.getSelectedFile();
16             String rutaEntrada = archivoEntrada.getAbsolutePath()
17                 ;
18
19             try {
20                 BuilderReader objReaderBuilder = new
21                     BuilderReader();
22                 Director objDirector = new Director(
23                     objReaderBuilder);
24
25                 objDirector.construir("avanzado", rutaEntrada);
26
27                 BufferedReader bufferedReader = objReaderBuilder.
28                     getBuffered();
29                 System.out.println("BufferedReader_construido:_"
30                     + rutaEntrada);
31                 bufferedReader.imprimirContenido();
32
33                 selector.setDialogTitle("destino_de_salida_Writer
34                     ");
35                 int seleccionEscritura = selector.showSaveDialog(
36                     null);
37
38                 if (seleccionEscritura == JFileChooser.
39                     APPROVE_OPTION) {
40                     File archivoSalida = selector.getSelectedFile
41                         ();
42                     String rutaSalida = archivoSalida.
43                         getAbsolutePath();
```

```
34
35         BuilderWriter objWriterBuilder = new
36             BuilderWriter();
37         objDirector.changeBuilder(objWriterBuilder);
38         objDirector.construir("simple", rutaSalida);
39
40         BufferedWriter bufferedWriter =
41             objWriterBuilder.getBuffered();
42         System.out.println("BufferedWriter construido
43             : " + rutaSalida);
44     }
45
46     } catch (Exception e) {
47         System.out.println("error en construccion: " + e
48             .getMessage());
49     }
50 }
else {
    System.out.println("Error");
}
```

Pruebas

Prueba de Lectura (BufferedReader)

A continuación se muestra la evidencia de la correcta instanciación del objeto `BufferedReader` y la extracción del texto utilizando el método `imprimirContenido()`:

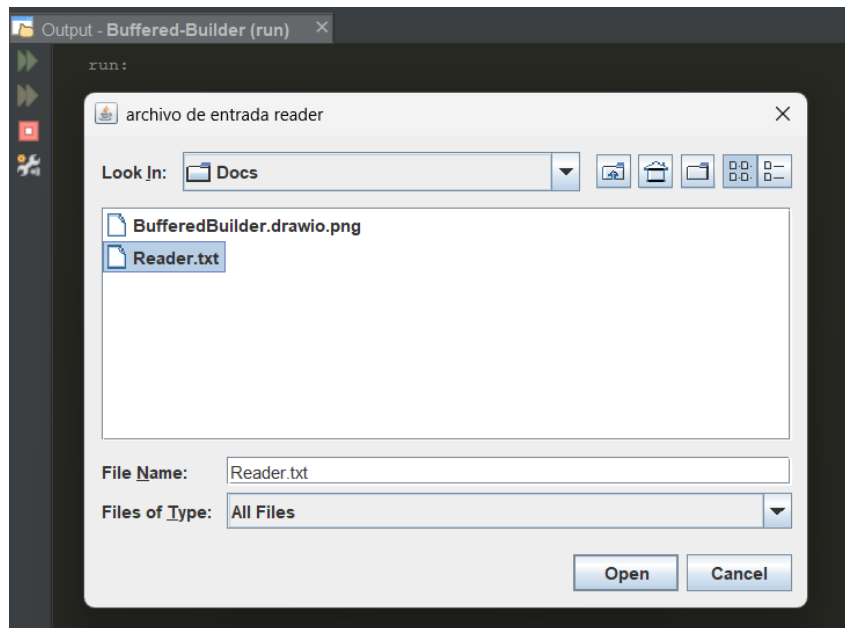


Figura 2: Jfile Chooser

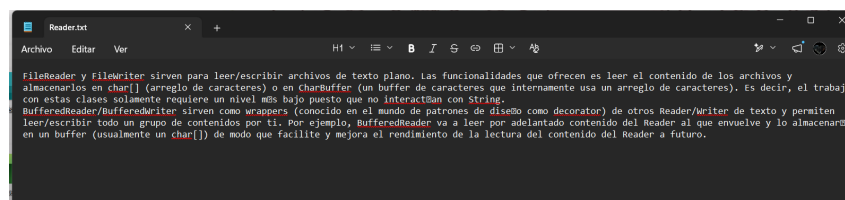
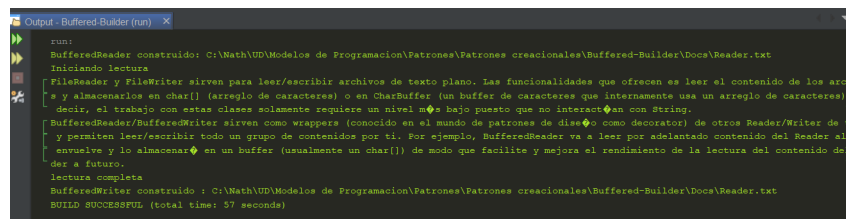


Figura 3: Archivo de texto txt.



```
run:
BufferedReader construido: C:\Nath\UD\Modelos de Programacion\Patrones\Patrones creacionales\Buffered-Builder\Docs\Reader.txt
Iniciando lectura
[
FileReader y FileWriter sirven para leer/escribir archivos de texto plano. Las funcionalidades que ofrecen es leer el contenido de los arch
s y almacenarlos en char[] (arreglo de caracteres) o en CharBuffer (un buffer de caracteres que internamente usa un arreglo de caracteres).
decir, el trabajo con estas clases solamente requiere un nivel m s bajo puesto que no interactúan con String.
BufferedReader/BufferedWriter sirven como wrappers (conocido en el mundo de patrones de dise o como decoratos) de otros Reader/Writer de t
y permiten leer/escribir todo un grupo de contenidos por ti. Por ejemplo, BufferedReader va a leer por adelantado contenido del Reader al
envuelve y lo almacena en un buffer (usualmente un char[]) de modo que facilite y mejora el rendimiento de la lectura del contenido del
der a futuro.
lectura completa
BufferedWriter construido : C:\Nath\UD\Modelos de Programacion\Patrones\Patrones creacionales\Buffered-Builder\Docs\Reader.txt
BUILD SUCCESSFUL (total time: 37 seconds)
```

Figura 4: Ejecución exitosa y lectura del archivo de entrada.

Prueba de Escritura (BufferedWriter)

Evidencia de la transición del Director utilizando el método `changeBuilder()` y la generación exitosa del `BufferedWriter` apuntando a la ruta de salida seleccionada:

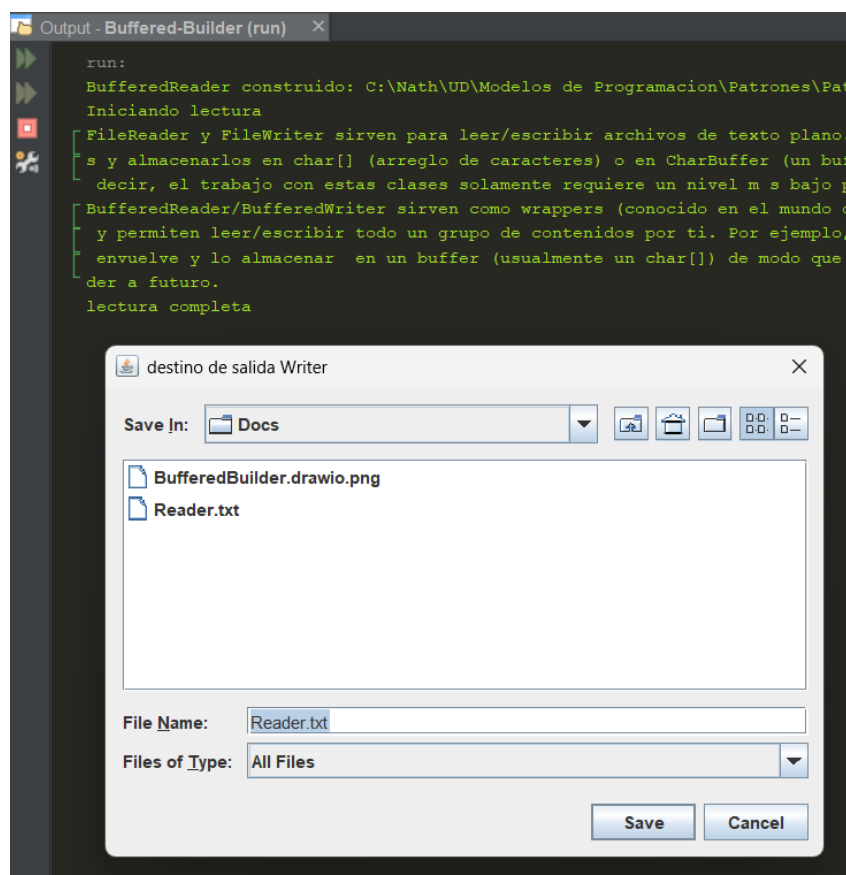


Figura 5: Jfile Chooser.

Estructura del Proyecto

Enlace al Repositorio

- <https://github.com/Thalyhx/Modelos-de-Programacion/tree/main/Patrones%20creacionales/Buffered-Builder>

Organización de Paquetes

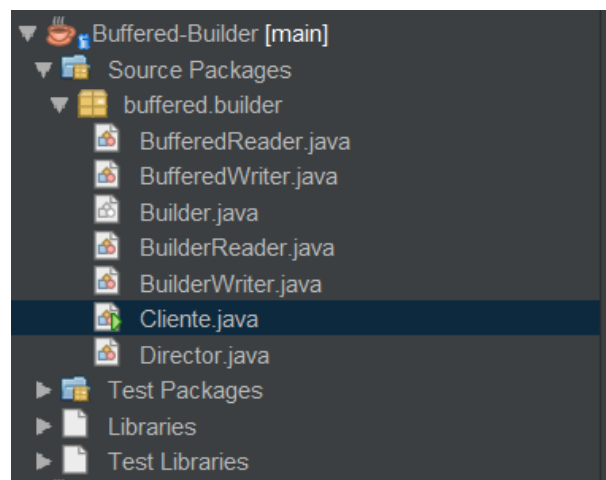


Figura 6: Estructura de paquetes en el IDE.